Пермский институт (филиал) федерального государственного бюджетного
образовательного учреждения высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Отделение информационных технологий и программирования

Практическая работа #1

Тема работы: «Модульное тестирование в Visual Studio (MSTest)»

Работу выполнил: Кошечкин Дмитрий

Группа: ИПс-21

Преподаватель: Берестов Дмитрий Борисович

Пермь 2026

Оглавление:

1. Цель работы 3

2. Описание проекта 3

3. Структура решения 3

4. Целевая платформа 3

5. Ход выполнения работы 4

5.1. Написание первого теста 4

5.2. Исправление кода 4

5.3. Добавление тестов для граничных условий 4

5.4. Рефакторинг кода и тестов 5

5.5. Финальный запуск тестов 5

6. Полученные результаты 5

7. Выводы 6

8. Ссылки 6

Отчёт по практической работе

«Модульное тестирование в Visual Studio (MSTest)»

Кошечкин Дмитрий · Группа ИПс-21

1. Цель работы

Освоить методику создания и выполнения модульных тестов для управляемого кода на платформе .NET с использованием MSTest в среде Visual Studio. Научиться выявлять и исправлять ошибки с помощью тестов, а также применять подход разработки на основе тестирования (TDD).

2. Описание проекта

Был разработан проект Bank, содержащий класс BankAccount, моделирующий банковский счёт. Класс предоставляет методы для списания (Debit), зачисления (Credit) и получения баланса. Исходно в методе Debit была допущена логическая ошибка: вместо вычитания суммы выполнялось сложение. Для проверки корректности работы создан тестовый проект BankTests с использованием фреймворка MSTest.

3. Структура решения

```
BankProject/
├── Bank.sln
├── Bank.slnx
├── Bank/
│   ├── Bank.csproj
│   └── BankAccount.cs
├── BankTests/
│   ├── BankTests.csproj
│   ├── BankAccountTests.cs
│   └── MSTestSettings.cs
└── Сопроводительная записка.pdf
```

4. Целевая платформа

Оба проекта ориентированы на .NET 8.0 (LTS). Используется тестовый фреймворк MSTest (MSTest.TestFramework и MSTest.TestAdapter) для среды Visual Studio 2022.

5. Ход выполнения работы

5.1. Написание первого теста

Был создан метод Debit_WithValidAmount_UpdatesBalance, который проверяет, что после списания допустимой суммы баланс уменьшается на заданную величину. При первом запуске тест завершился ошибкой, так как в коде использовалось сложение вместо вычитания.

```
[TestMethod]
public void Debit_WithValidAmount_UpdatesBalance()
{
    double beginningBalance = 11.99;
    double debitAmount = 4.55;
    double expected = 7.44;
    var account = new BankAccount("Mr. Bryan Walton", beginningBalance);
    account.Debit(debitAmount);
    Assert.AreEqual(expected, account.Balance, 0.001, "Account not debited correctly");
}
```

5.2. Исправление кода

В методе Debit класса BankAccount строка m_balance += amount; была заменена на m_balance -= amount;. После этого тест стал проходить успешно.

5.3. Добавление тестов для граничных условий

Были добавлены два теста, проверяющие выброс исключений при некорректных аргументах:

• Debit_WhenAmountIsLessThanZero_ShouldThrowArgumentOutOfRange
• Debit_WhenAmountIsMoreThanBalance_ShouldThrowArgumentOutOfRange

Тесты реализованы через конструкцию try/catch с проверкой точного текста сообщения исключения:

```
[TestMethod]
public void Debit_WhenAmountIsLessThanZero_ShouldThrowArgumentOutOfRange()
{
    double beginningBalance = 11.99;
    double debitAmount = -100.00;
    BankAccount account = new BankAccount("Mr. Bryan Walton", beginningBalance);
    try
    {
        account.Debit(debitAmount);
        Assert.Fail("Expected exception was not thrown");
    }
    catch (System.ArgumentOutOfRangeException e)
    {
        StringAssert.Contains(e.Message, BankAccount.DebitAmountLessThanZeroMessage);
    }
}
```

5.4. Рефакторинг кода и тестов

В класс BankAccount были добавлены публичные константы для сообщений об ошибках:

```
public const string DebitAmountExceedsBalanceMessage = "Debit amount exceeds balance";
public const string DebitAmountLessThanZeroMessage = "Debit amount is less than zero";
public const string CreditAmountLessThanZeroMessage = "Credit amount is less than zero";
```

Конструктор исключения в методе Debit был заменён на версию с подробным сообщением:

```
throw new ArgumentOutOfRangeException(nameof(amount), amount,
    DebitAmountLessThanZeroMessage);
```

Тесты были дополнены проверкой корректности сообщения через StringAssert.Contains, что повысило надёжность проверок.

5.5. Финальный запуск тестов

После всех исправлений все тесты успешно проходят в Обозревателе тестов Visual Studio 2022 и через интерфейс командной строки .NET CLI:

```
> dotnet test BankTests/BankTests.csproj
Исходные файлы: BankTests.dll (.NETCoreApp,Version=v8.0)
[MSTest][Discovery] Обнаружено тестов: 5
[MSTest][Run] Пройден: Debit_WithValidAmount_UpdatesBalance
[MSTest][Run] Пройден: Debit_WhenAmountIsLessThanZero_ShouldThrowArgumentOutOfRange
[MSTest][Run] Пройден: Debit_WhenAmountIsMoreThanBalance_ShouldThrowArgumentOutOfRange
[MSTest][Run] Пройден: Credit_WithValidAmount_UpdatesBalance
[MSTest][Run] Пройден: Credit_WhenAmountIsLessThanZero_ShouldThrowArgumentOutOfRange
Успешно! Пройдено: 5, Сбоев: 0, Пропущено: 0.
```

6. Полученные результаты

Разработан класс BankAccount с корректной логикой списания и пополнения средств. Создан набор модульных тестов, покрывающих:

1. Основной сценарий (успешное списание допустимой суммы);
2. Попытку списания отрицательной суммы;
3. Попытку списания суммы, превышающей баланс;
4. Корректное зачисление средств методом Credit;
5. Обработку исключения при зачислении отрицательной суммы.

Все тесты выполняются успешно, что подтверждает корректность реализации методов класса BankAccount.

7. Выводы

В ходе работы были освоены:

• создание тестовых проектов в среде Visual Studio 2022;
• написание и запуск модульных тестов с помощью фреймворка MSTest;
• использование утверждений (Assert, StringAssert);
• обработка и проверка исключительных ситуаций в тестах;
• подход «красный-зелёный-рефакторинг» (TDD).

Модульное тестирование позволило своевременно выявить и устранить логическую ошибку в методе Debit, а также обеспечить надёжную защиту от регрессий при дальнейших изменениях кода решения.

8. Ссылки

1. Исходный код проекта: GitHub репозиторий: https://github.com/guzlik12311/Koshechkin-Bank-Peresdacha
2. Документация MSTest: https://learn.microsoft.com/ru-ru/dotnet/core/testing/unit-testing-with-mstest
3. Пошаговое руководство Microsoft Learn: Создание и запуск модульных тестов для управляемого кода: https://learn.microsoft.com/ru-ru/visualstudio/test/walkthrough-creating-and-running-unit-tests-for-managed-code